S.HARISH (192521003)

1. A university aims to develop a Student Analytics System to evaluate academic performance across multiple departments, where each department has a varying number of students and their marks are stored in a two-dimensional structure. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent and manage student data effectively.

A. The program should compute the topper in each department, identify the overall university topper, and calculate the pass percentage for each department.

B. Additionally, the solution must handle variable department sizes, edge cases such as empty departments, equal marks, or all students failing, and ensure efficient comparison across departments while maintaining optimized time complexity and clean modular design.

Code:

```
#include <iostream>
using namespace std;

class Student
{
private:
   string name;
   int marks;

public:
   void input()
```

```
  {
    cin >> name >> marks;
  }

  int getMarks()
  {
    return marks;
  }

  string getName()
  {
    return name;
  }
};

class Department
{
private:
  string deptName;
  int n;

public:
  Student s[100];

  void input()
  {
    cout<<"Department Name: ";
    cin>>deptName;
```

```
    cout<<"Number of Students: ";
    cin>>n;

    for(int i=0;i<n;i++)
        s[i].input();
}

void topper()
{
    if(n==0)
    {
        cout<<"Empty Department\n";
        return;
    }

    int maxMarks=s[0].getMarks();
    string topper=s[0].getName();

    for(int i=1;i<n;i++)
    {
        if(s[i].getMarks()>maxMarks)
        {
            maxMarks=s[i].getMarks();
            topper=s[i].getName();
        }
    }
```

```
        cout<<"Topper: "<<topper<<" "<<maxMarks<<endl;
    }

    float passPercentage()
    {
        if(n==0)
            return 0;

        int pass=0;

        for(int i=0;i<n;i++)
        {
            if(s[i].getMarks()>=40)
                pass++;
        }

        return (pass*100.0)/n;
    }
};

int main()
{
    int d;
    cin>>d;

    Department dept[10];

    for(int i=0;i<d;i++)
```

```
    dept[i].input();

  for(int i=0;i<d;i++)
  {
    dept[i].topper();
    cout<<"Pass % = "<<dept[i].passPercentage()<<endl;
  }

  return 0;
}
```

OUTPUT:

Test Cases

Case 1 (BEST CASE)

Input:

Departments = 1

IT

Students:

Harish 80

Ravi 90

John 70

Output:

Topper: Ravi 90

Pass % = 100%

CASE 2 (WORST CASE)

Departments:1

CSE

Students:3

Harish 20

Ravi 35

John 10

Output:

Topper: Ravi 35

Pass % = 0

CASE 3(EMPTY STUDENT)

Departments: 1

CSE

Students:0

Output:

Empty Department

Pass % = 0

2. A country conducts elections across multiple regions, where votes for different candidates are stored in a two-dimensional structure representing regions and candidates. Develop a C++ program using object-oriented principles and functions to store and process vote data efficiently.

A. The program should determine the winner in each region, compute the overall winner by aggregating votes across all regions, and detect tie conditions at both regional and overall levels.

B. The implementation should use appropriate access specifiers, member functions, and static members where necessary, and must handle large input sizes efficiently. Additionally, the program should address edge cases such as ties, zero votes, and invalid inputs, while ensuring accurate multi-level comparisons and optimized performance.

Code:

#include<iostream>

```
using namespace std;

class Election
{
public:
   static int totalVotes;
   static int maxVotes;
   static int winnerIndex;

   void process(int votes[][50], int regions, int candidates)
   {
      int total[50] = {0};

      for(int j = 0; j < candidates; j++)
      {
         for(int i = 0; i < regions; i++)
         {
            total[j] += votes[i][j];
            totalVotes += votes[i][j];
         }
         if(total[j] > maxVotes)
         {
            maxVotes = total[j];
            winnerIndex = j;
         }
      }
      cout << "\n===== RESULT =====\n";
      cout << "Total Votes = " << totalVotes << endl;
```

```
        cout << "Winner Candidate Index = " << winnerIndex + 1 << endl;
        cout << "Max Votes = " << maxVotes << endl;
    }
};
int Election::totalVotes = 0;
int Election::maxVotes = 0;
int Election::winnerIndex = 0;
int main()
{
    int regions, candidates;

    cout << "Enter number of regions: ";
    cin >> regions;

    cout << "Enter number of candidates: ";
    cin >> candidates;

    int votes[50][50];

    cout << "\nEnter votes (region-wise):\n";

    for(int i = 0; i < regions; i++)
    {
        cout << "Region " << i + 1 << ":\n";

        for(int j = 0; j < candidates; j++)
        {
            cout << "Candidate " << j + 1 << ": ";
```

```
            cin >> votes[i][j];
        }
    }

    Election e;
    e.process(votes, regions, candidates);

    return 0;
}
```

TEST CASE(BEST CASE)

Input

Enter number of regions: 2

Enter number of candidates: 3

Region 1:

100 150 120

Region 2:

200 100 180

Output

Total Votes = 850

Winner Candidate Index = 1

Max Votes = 300

TEST CASE (WORST CASE)

Input

Enter number of regions: 2

Enter number of candidates: 3

Region 1:

0 0 0

Region 2:

0 0 0

_____

Output

Total Votes = 0

There is no winner candidate

Max Votes = 0

In this scenario, a Retail Chain Management System is designed to analyze product sales across multiple stores using object-oriented programming in C++.

3)To manage multiple stores, the program can either use a two-dimensional array of objects or create another class such as Store, which contains an array of Product objects. For each store, the total sales are calculated by

iterating through all its products and summing their individual sales using a dedicated member function like calculateStoreSales(). A static data member is used to maintain the overall total sales across all stores, demonstrating the use of shared class-level data. This helps in tracking cumulative performance of the entire retail chain.

C. The program also includes logic to identify the best-performing store, which is the store with the highest total sales, and the best-selling product, which is the product generating the highest revenue across all stores.

D. Proper input/output handling is essential, where the user is prompted to enter the number of stores, number of products per store, and corresponding product details. The output should be well-structured, displaying store-wise sales, overall sales, best-performing store, and top product.Code:

```
#include<iostream>
using namespace std;

class Product
{
public:
  string name;
  float price;
  int qty;

  void input()
  {
    cout << "Enter product name, price, quantity: ";
    cin >> name >> price >> qty;
  }

  float sales()
  {
    return price * qty;
  }
};

class Store
{
```

```
public:
  Product p[100];
  int n;

  static float overallSales;
  static string bestProduct;
  static float maxProductSales;

  void input()
  {
    cout << "\nEnter number of products in this store: ";
    cin >> n;

    for(int i = 0; i < n; i++)
      p[i].input();
  }

  float calculateStoreSales()
  {
    float total = 0;

    for(int i = 0; i < n; i++)
    {
      float s = p[i].sales();
      total += s;

      // BEST PRODUCT CHECK
      if(s > maxProductSales)
```

```
        {
            maxProductSales = s;
            bestProduct = p[i].name;
        }
      }

      overallSales += total;
      return total;
   }
};

// STATIC INITIALIZATION
float Store::overallSales = 0;
float Store::maxProductSales = 0;
string Store::bestProduct = "";

int main()
{
   int stores;

   cout << "Enter number of stores: ";
   cin >> stores;

   Store s[50];

   float bestStoreSales = 0;
   int bestStoreIndex = 0;
```

```
    for(int i = 0; i < stores; i++)
    {
        cout << "\n===== STORE " << i + 1 << " =====\n";

        s[i].input();

        float total = s[i].calculateStoreSales();

        cout << "Store " << i + 1 << " Sales = " << total << endl;

        if(total > bestStoreSales)
        {
            bestStoreSales = total;
            bestStoreIndex = i;
        }
    }

    cout << "\n===== FINAL RESULT =====\n";
    cout << "Best Store: " << bestStoreIndex + 1 << endl;
    cout << "Best Store Sales: " << bestStoreSales << endl;
    cout << "Best Selling Product: " << Store::bestProduct << endl;
    cout << "Overall Sales: " << Store::overallSales << endl;

    return 0;
}
```

TEST CASE 1 (NORMAL CASE)

Input

Enter number of stores: 2

Store 1:

3

Laptop 50000 2

Mouse 1000 5

Keyboard 2000 2

Store 2:

2

Phone 30000 3

Headset 2000 2

Calculation

Store 1:

- Laptop = 100000
- Mouse = 5000
- Keyboard = 4000

Total = 109000

Store 2:

- Phone = 90000
- Headset = 4000

Total = 94000

Output

Store 1 Sales = 109000

Store 2 Sales = 94000

Best Store: 1

Best Store Sales: 109000

Best Selling Product: Laptop

Overall Sales: 203000

Best Case Test Case

Input:

Enter product name, price, quantity:

Pen 10 5

Expected Output:

Sales = 50

Worst Case Test Case

Input:

Enter product name, price, quantity:

Laptop 99999 1000

Expected Output:

Sales = 99999000